SIS-2: Predicting Diabetes with the CDC Diabetes Health Indicators Dataset

Dataset: CDC Diabetes Health Indicators (UCI #891), the BRFSS 2015 survey file `diabetes_binary_health_indicators_BRFSS2015.csv`.

- **Rows:** 253,680 survey respondents.
- **Features:** 21 health, lifestyle and demographic variables (e.g. `HighBP`, `BMI`, `Age`, `Income`).
- **Target:** `Diabetes_binary` (0 = no diabetes, 1 = prediabetes or diabetes).

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
```

EDA

In [3]:
```
df = pd.read_csv('/Users/elzaniyazbekova/Desktop/MLLM/untitled folder/dia
```

In [4]:
```
df.head()
```

Out[4]:

	Diabetes_binary	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDis
0	0.0	1.0	1.0	1.0	40.0	1.0	0.0	
1	0.0	0.0	0.0	0.0	25.0	1.0	0.0	
2	0.0	1.0	1.0	1.0	28.0	0.0	0.0	
3	0.0	1.0	0.0	1.0	27.0	0.0	0.0	
4	0.0	1.0	1.0	1.0	24.0	0.0	0.0	

5 rows × 22 columns

In [5]:
```
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (253680, 22)
```

In [6]:
```
print("Features")
print(list(df.columns))
```

```
Features
['Diabetes_binary', 'HighBP', 'HighChol', 'CholCheck', 'BMI', 'Smoker', 'S
troke', 'HeartDiseaseorAttack', 'PhysActivity', 'Fruits', 'Veggies', 'HvyA
lcoholConsump', 'AnyHealthcare', 'NoDocbcCost', 'GenHlth', 'MentHlth', 'Ph
ysHlth', 'DiffWalk', 'Sex', 'Age', 'Education', 'Income']
```

In [7]:
```
df.describe()
```

Out[7]:

	Diabetes_binary	HighBP	HighChol	CholCheck	
count	253680.000000	253680.000000	253680.000000	253680.000000	253680.0(
mean	0.139333	0.429001	0.424121	0.962670	28.3{
std	0.346294	0.494934	0.494210	0.189571	6.6(
min	0.000000	0.000000	0.000000	0.000000	12.0(
25%	0.000000	0.000000	0.000000	1.000000	24.0(
50%	0.000000	0.000000	0.000000	1.000000	27.0(
75%	0.000000	1.000000	1.000000	1.000000	31.0(
max	1.000000	1.000000	1.000000	1.000000	98.0(

8 rows × 22 columns

Dataset overview

- The dataset has 253,680 rows and 22 columns (21 features + the target `Diabetes_binary`).
- All columns are numeric (float). `df.info()` below confirms there are no missing values, so no imputation is needed.

In [42]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 253680 entries, 0 to 253679
Data columns (total 22 columns):
 #   Column                Non-Null Count   Dtype  
---  ------                --------------   -----  
 0   Diabetes_binary       253680 non-null  float64
 1   HighBP                253680 non-null  float64
 2   HighChol              253680 non-null  float64
 3   CholCheck             253680 non-null  float64
 4   BMI                   253680 non-null  float64
 5   Smoker                253680 non-null  float64
 6   Stroke                253680 non-null  float64
 7   HeartDiseaseorAttack  253680 non-null  float64
 8   PhysActivity          253680 non-null  float64
 9   Fruits                253680 non-null  float64
 10  Veggies               253680 non-null  float64
 11  HvyAlcoholConsump     253680 non-null  float64
 12  AnyHealthcare         253680 non-null  float64
 13  NoDocbcCost           253680 non-null  float64
 14  GenHlth               253680 non-null  float64
 15  MentHlth              253680 non-null  float64
 16  PhysHlth              253680 non-null  float64
 17  DiffWalk              253680 non-null  float64
 18  Sex                   253680 non-null  float64
 19  Age                   253680 non-null  float64
 20  Education             253680 non-null  float64
 21  Income                253680 non-null  float64
dtypes: float64(22)
memory usage: 42.6 MB
```

In [37]:
```
ordinal_cols = ['GenHlth', 'Age', 'Education', 'Income']
num_cols   = ['BMI', 'MentHlth', 'PhysHlth']
binary_cols  = ['HighBP','HighChol','CholCheck','Smoker','Stroke','HeartD

df[binary_cols + ordinal_cols + num_cols].nunique().sort_values()
```

Out[37]:
```
HighBP                 2
Sex                    2
DiffWalk               2
NoDocbcCost            2
HvyAlcoholConsump      2
Veggies                2
Fruits                 2
AnyHealthcare          2
HeartDiseaseorAttack   2
Stroke                 2
Smoker                 2
CholCheck              2
HighChol               2
PhysActivity           2
GenHlth                5
Education              6
Income                 8
Age                   13
MentHlth              31
PhysHlth              31
BMI                   84
dtype: int64
```

Feature types

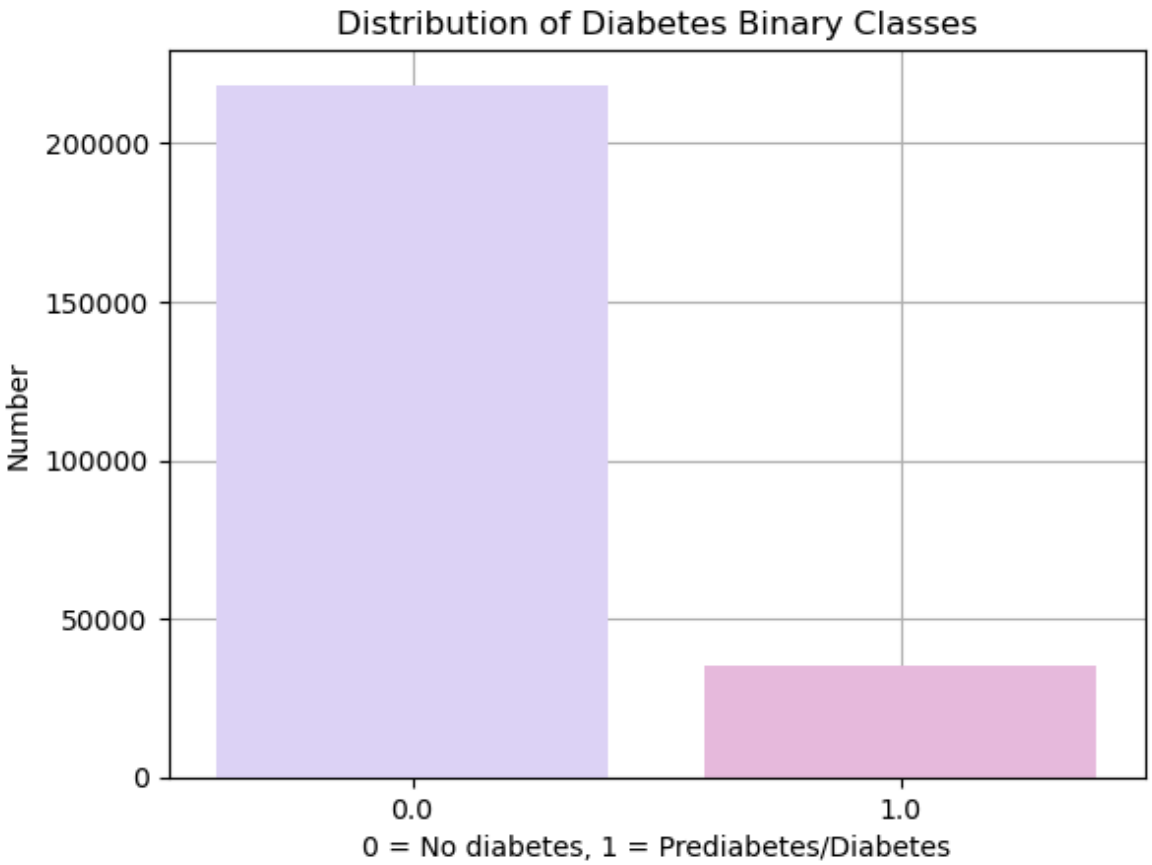
- **14 binary flags** (e.g. `HighBP`, `Smoker`, `Sex`, `DiffWalk`) are already encoded 0/1.
- **4 ordinal features** (`GenHlth` 1–5, `Age` 1–13 age bands, `Education` 1–6, `Income` 1–8) have a natural order, so they are kept as integers rather than one-hot encoded.
- **3 numeric features** (`BMI`, `MentHlth`, `PhysHlth`) are on very different scales (BMI ≈ 12–98 vs 0/1 flags), which matters for distance- and gradient-based models.

In [9]:
```
print("\nTarget Distribution:")
df['Diabetes_binary'].value_counts()
```

```
Target Distribution:
```

Out[9]:
```
Diabetes_binary
0.0    218334
1.0     35346
Name: count, dtype: int64
```

In [10]:
```
# plt.grid(axis='y', linestyle='-')
sns.countplot(x='Diabetes_binary', data=df, palette=['#DCD0FF', '#F0B2E0'
plt.gca().set_axisbelow(True)
plt.title('Distribution of Diabetes Binary Classes')
plt.xlabel('0 = No diabetes, 1 = Prediabetes/Diabetes')
plt.ylabel('Number')
plt.show()
```



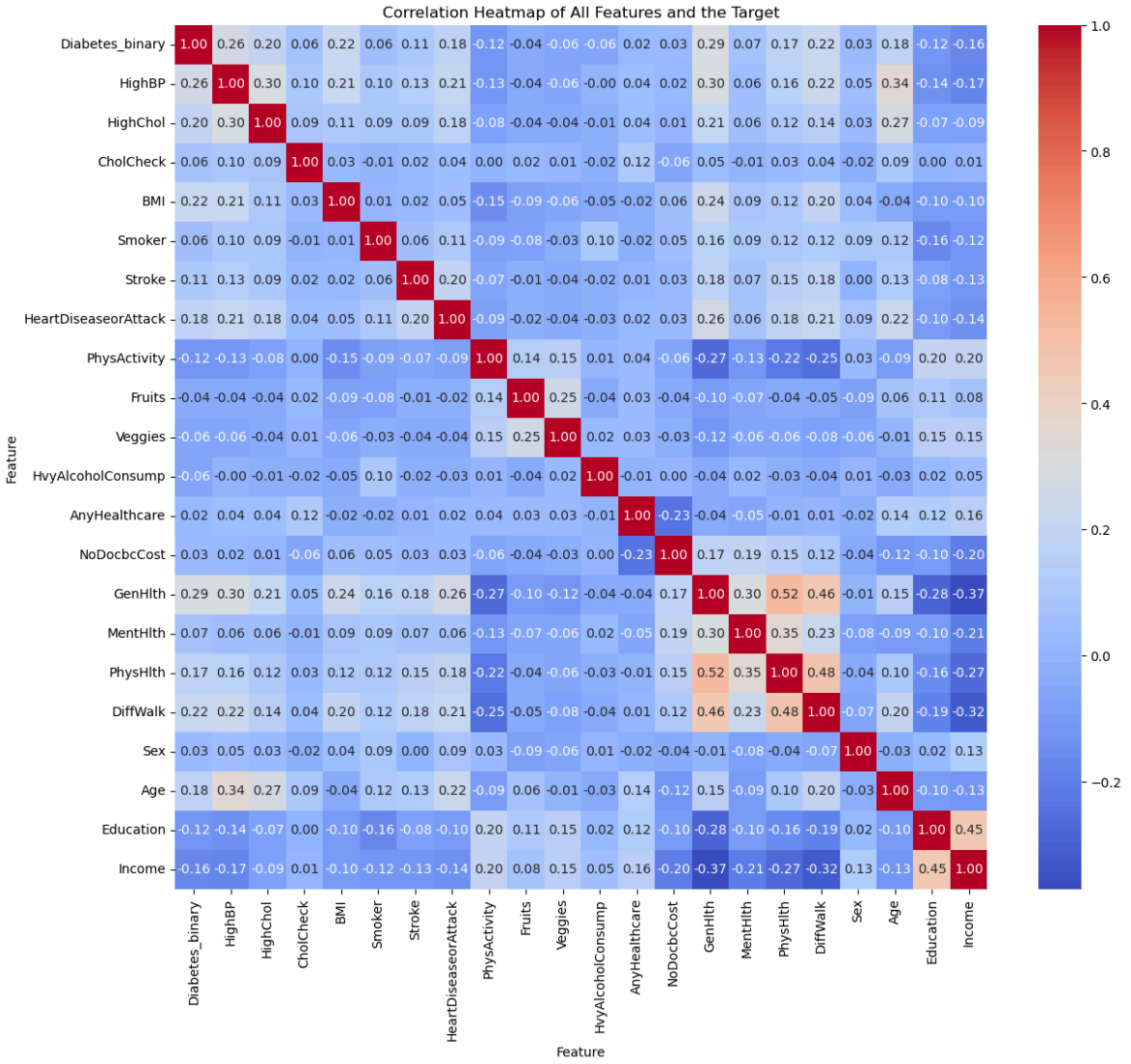
Interpretation: class imbalance

- The data is strongly imbalanced. About 86% of respondents are non-diabetic (class 0) and about 14% are diabetic (class 1)
- A model that always predicts "no diabetes" would score about 86% accuracy while detecting no diabetic patients. So we will (1) use a **stratified** split, (2) evaluate with **recall, F1, PR-AUC and ROC-AUC** rather than accuracy, and (3) handle the imbalance with class weights.

In [11]:
```
plt.figure(figsize=(14, 12))

c = df.corr()

sns.heatmap(c, cmap="coolwarm", annot=True, fmt='.2f')
plt.title('Correlation Heatmap of All Features and the Target')
plt.xlabel('Feature'); plt.ylabel('Feature')
plt.show()
```



Interpretation

- No feature is strongly correlated with the target.

- The highest correlations (~0.2–0.4) are for `GenHlth`, `HighBP`, `BMI`, `HighChol`, `Age`, `DiffWalk` and `PhysHlth`, so no single variable is enough and a multi-feature model is needed.
- `PhysHlth` – `GenHlth`, `DiffWalk` – `PhysHlth` and `Income` – `Education` show moderate inter-feature correlation, but none is high enough (> 0.8) to require dropping a feature for multicollinearity.

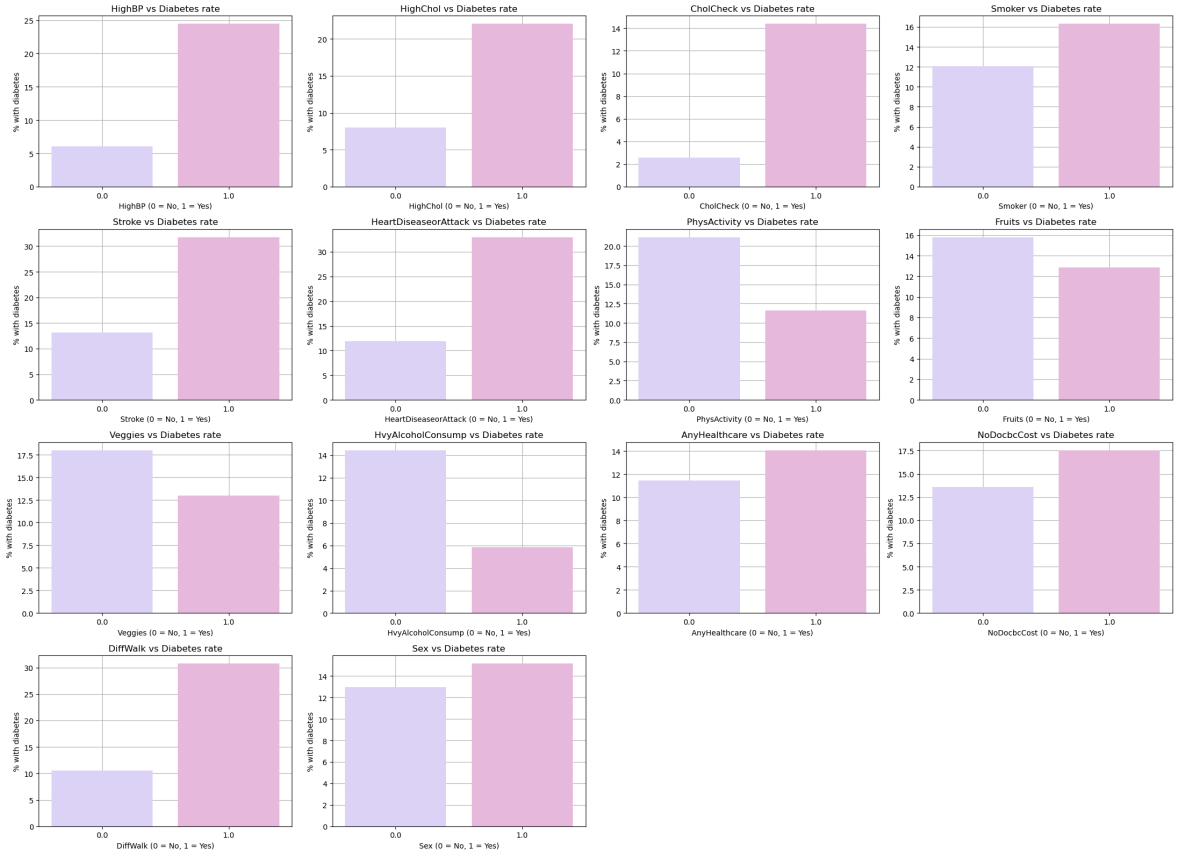
In [38]:
```
plt.figure(figsize=(22, 16))
for i, col in enumerate(binary_cols):


    plt.subplot(4, 4, i + 1)
    rate = df.groupby(col)['Diabetes_binary'].mean() * 100
    sns.barplot(x=rate.index, y=rate.values, palette=['#DCD0FF', '#F0B2E0

    plt.title(f'{col} vs Diabetes rate')
    plt.xlabel(f'{col} (0 = No, 1 = Yes)')
    plt.ylabel('% with diabetes')
    plt.grid()
    plt.gca().set_axisbelow(True)


plt.tight_layout()

plt.show()
```



In [39]:
```
plt.figure(figsize=(20, 15))

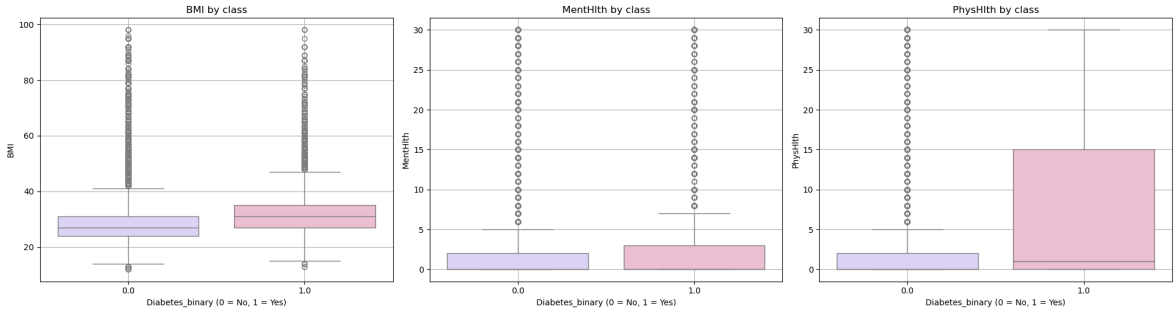
for i, col in enumerate(num_cols):
    plt.subplot(3, 3, i + 1)

    sns.boxplot(data=df, x='Diabetes_binary', y=col, hue='Diabetes_binary
```

```
                palette=['#DCD0FF', '#F4B6D2'], legend=False)

    plt.title(f'{col} by class')
    plt.ylabel(col)
    plt.xlabel('Diabetes_binary (0 = No, 1 = Yes)')
    plt.grid()
    plt.gca().set_axisbelow(True)

plt.tight_layout()
plt.show()
```



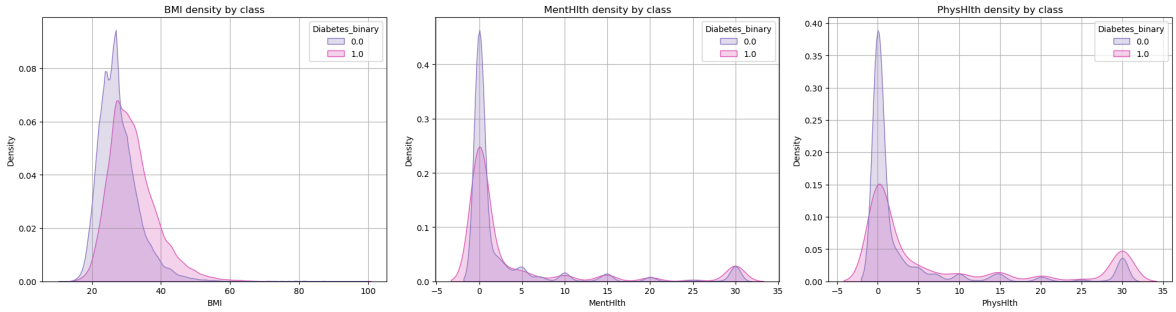
In [14]:
```
plt.figure(figsize=(20, 15))

for i, col in enumerate(num_cols):
    plt.subplot(3, 3, i + 1)

    sns.kdeplot(data=df, x=col, hue='Diabetes_binary',
                common_norm=False, fill=True,
                palette=['#8E7CC3', '#E255B6'])

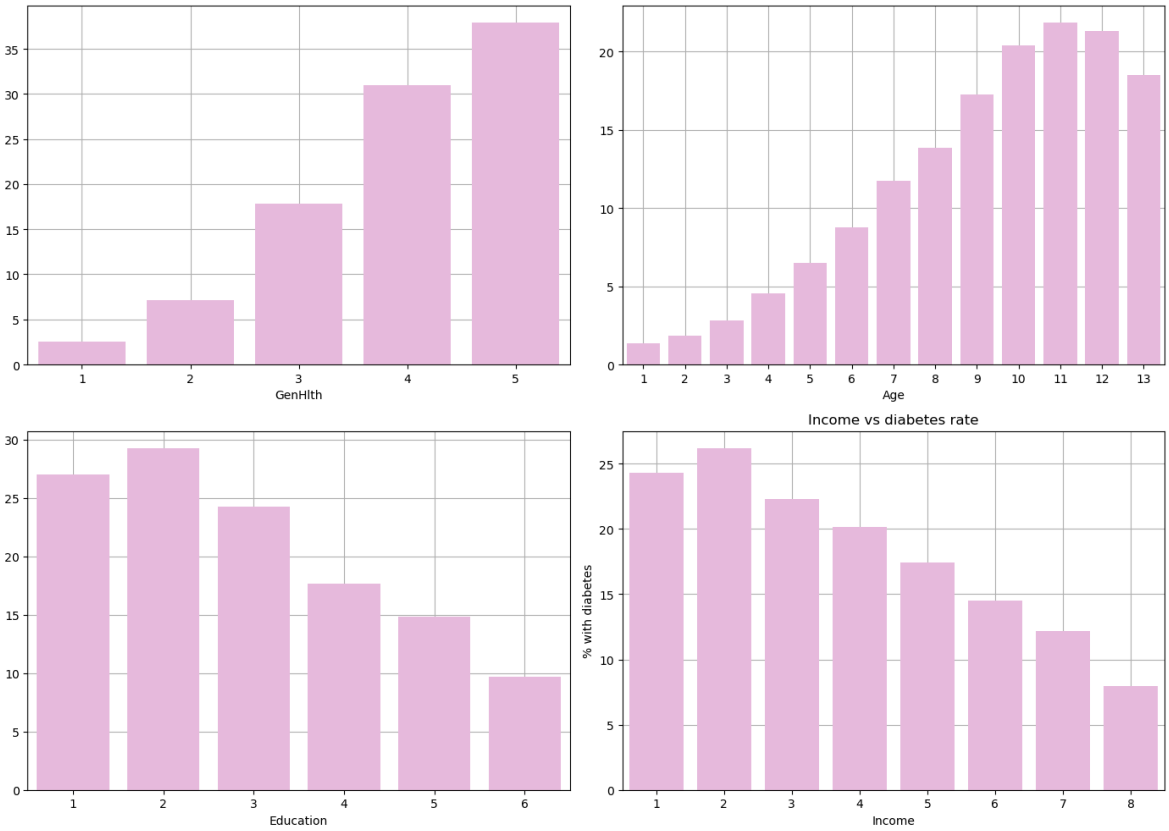
    plt.title(f'{col} density by class')
    plt.ylabel("Density")
    plt.xlabel(col)
    plt.grid()
    plt.gca().set_axisbelow(True)

plt.tight_layout()
plt.show()
```



In [64]:
```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
for ax, col in zip(axes.ravel(), ordinal_cols):
    rate = df.groupby(col)['Diabetes_binary'].mean() * 100
    sns.barplot(x=rate.index.astype(int), y=rate.values, color='#F0B2E0',
    ax.grid()
    ax.set_axisbelow(True)
    plt.title(f'{col} vs diabetes rate')
    plt.xlabel(col)
    plt.ylabel('% with diabetes')
```

```
plt.tight_layout()
plt.show()
print("Caption: Diabetes rate rises with worse general health and older a
```




```
Caption: Diabetes rate rises with worse general health and older age and f
alls with higher income and education.
```

Interpretation

Features whose two bars differ most are the most informative.

- `HighBP`, `HighChol`, `HeartDiseaseorAttack`, `DiffWalk` and `Stroke` show a much higher diabetes rate when the value is 1.
- `PhysActivity`, `Fruits` and `Veggies` show a lower rate when the value is 1.
- `AnyHealthcare`, `NoDocbcCost`, `Smoker` and `Sex` differ only slightly, so they are likely weak predictors.
- `HvyAlcoholConsump` is lower for heavy drinkers, likely because of confounding (for example, people diagnosed with diabetes drink less). This is an association, not a causal effect.

Data Preprocessing

In [15]:
```
df.isnull().sum()
```

Out[15]:
```
Diabetes_binary         0
HighBP                  0
HighChol                0
CholCheck               0
BMI                     0
Smoker                  0
Stroke                  0
HeartDiseaseorAttack    0
PhysActivity            0
Fruits                  0
Veggies                 0
HvyAlcoholConsump       0
AnyHealthcare           0
NoDocbcCost             0
GenHlth                 0
MentHlth                0
PhysHlth                0
DiffWalk                0
Sex                     0
Age                     0
Education               0
Income                  0
dtype: int64
```

In [16]:
```
n_dup = df.duplicated().sum()
print(f"Duplicate rows: {n_dup}")
```

```
Duplicate rows: 24206
```

There are no missing values, so imputation (mean/median/mode, KNN imputer) is not needed. There are 24,206 duplicate rows (9.5%). They are respondents with identical answers on low-cardinality variables, not necessarily data-entry errors, so they are kept. Removing them changes the positive rate from 13.9% to 15.3%.

In [17]:
```
X = df.drop('Diabetes_binary', axis=1)
y = df['Diabetes_binary']
```

In [18]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
 X, y,
 test_size=0.2,
 random_state=42, stratify=y
)
print("Train", X_train.shape,"Test", X_test.shape)

lo, hi = X_train['BMI'].quantile([0.01, 0.99])

X_train, X_test = X_train.copy(), X_test.copy()
X_train['BMI'] = X_train['BMI'].clip(lo, hi)
X_test['BMI']  = X_test['BMI'].clip(lo, hi)
```

```
Train (202944, 21) Test (50736, 21)
```

Diabetes prevalence increases steadily as `GenHlth` gets worse (1 = excellent, 5 = poor) and with age band, and decreases with higher `Income` and `Education`. The raw BMI distribution is right-skewed with a long tail up to 98; clipping at the 1st and 99th percentile removes extreme values without changing the main shape.

In [19]:
```
from sklearn.preprocessing import  StandardScaler

numerical_preprocessor = StandardScaler()


from sklearn.compose import ColumnTransformer

num1_cols = df.select_dtypes(include="number").drop(columns=['Diabetes_bi
preprocessor = ColumnTransformer([
    ('standard-scaler', numerical_preprocessor, list(num1_cols))])
X_train_s = preprocessor.fit_transform(X_train)
X_test_s  = preprocessor.transform(X_test)
```

Split first, with `stratify=y`, so both sets keep the ~86/14 class ratio. `StandardScaler` (mean 0, std 1) is fit on the training data only and then applied to the test set. This prevents leakage. Scaling matters for Logistic Regression, KNN and SVM. Tree models such as Decision Tree, Random Forest and LightGBM don't need it, but it does no harm. Binary flags are left as 0/1.

Model Training

Model	Imbalance handling	Notes
Logistic Regression	`class_weight='balanced'`	linear, interpretable baseline
Decision Tree	`class_weight='balanced'`, entropy criterion	unpruned, so it will overfit
Random Forest	`class_weight='balanced'`	bagged trees
LightGBM	`class_weight='balanced'`	gradient boosting
XGBoost	`scale_pos_weight` = n(neg)/n(pos) ≈ 6.2	gradient boosting
KNN	none (k = 3, distance-weighted)	distance-based, sensitive to scaling
SVM (RBF)	`class_weight='balanced'`	trained on a 30,000-row stratified subsample because SVM training scales roughly quadratically with n

In [20]:
```
# Logistic Regression, Decision Tree/ Random Forest,
# # XGBoost/LightGBM, KNN, SVM
```

In [21]:
```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(max_iter=1000,
        class_weight='balanced', random_state=42)
log_reg.fit(X_train_s, y_train)
```

Out[21]:

▾ LogisticRegression

▶ Parameters

In [22]:
```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    class_weight='balanced', criterion="entropy", random_state=42)
dt = dt.fit(X_train_s, y_train)
```

In [23]:
```
rf_classifier = RandomForestClassifier(class_weight='balanced', random_st
rf_classifier.fit(X_train_s, y_train)
```

Out[23]:

▾ RandomForestClassifier

▶ Parameters

In [24]:
```
import lightgbm as lgb
```

In [25]:
```
clf = lgb.LGBMClassifier(class_weight='balanced', random_state=42)
clf.fit(X_train_s, y_train)
```

```
[LightGBM] [Info] Number of positive: 28277, number of negative: 174667
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.004558 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 176
[LightGBM] [Info] Number of data points in the train set: 202944, number o
f used features: 21
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.00
0000
[LightGBM] [Info] Start training from score 0.000000
```

Out[25]:

▾ LGBMClassifier

▶ Parameters

In [26]:
```
from xgboost import XGBClassifier
scale_pos = (y_train == 0).sum() / (y_train == 1).sum()
xgb=XGBClassifier(scale_pos_weight=scale_pos, random_state=42)
xgb.fit(X_train_s, y_train)
```

Out[26]:

▾ XGBClassifier

▶ Parameters

In [27]:
```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3, weights='distance')

knn.fit(X_train_s, y_train)
```

Out[27]:

▾ KNeighborsClassifier

▶ Parameters

In [28]:
```
from sklearn.svm import SVC

X_svm, _, y_svm, _ = train_test_split(X_train_s, y_train, train_size=3000
                                      stratify=y_train, random_state=42)
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', class_weight='balance
svm_model.fit(X_svm, y_svm)
```

Out[28]:

▾ SVC

▶ Parameters

On 200,000 rows an RBF SVM would take hours. Sample is stratified, so it keeps the 86/14 class ratio.

In [69]:
```
from sklearn.model_selection import StratifiedKFold, cross_val_score

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_models = {'Logistic Regression': log_reg, 'LightGBM': clf, 'XGBoost':
             'Random Forest': rf_classifier, 'Decision Tree': dt}
cv_rows = []
for name, m in cv_models.items():
    s = cross_val_score(m, X_train_s, y_train, cv=skf, scoring='roc_auc',
    cv_rows.append({'Model': name, 'CV ROC-AUC mean': s.mean(), 'std': s.
pd.DataFrame(cv_rows).sort_values('CV ROC-AUC mean', ascending=False)
```

```
[LightGBM] [Info] Number of positive: 22622, number of negative: 139734
[LightGBM] [Info] Number of positive: 22622, number of negative: 139733
[LightGBM] [Info] Number of positive: 22621, number of negative: 139734
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.012421 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 176
[LightGBM] [Info] Number of data points in the train set: 162356, number o
f used features: 21
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=-0.0
00000
[LightGBM] [Info] Start training from score -0.000000
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.010185 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 176
[LightGBM] [Info] Number of data points in the train set: 162355, number o
f used features: 21
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.009925 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 176
[LightGBM] [Info] Number of data points in the train set: 162355, number o
f used features: 21
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.00
0000
[LightGBM] [Info] Start training from score 0.000000
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.00
0000
[LightGBM] [Info] Start training from score 0.000000
[LightGBM] [Info] Number of positive: 22621, number of negative: 139734
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.004705 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 176
[LightGBM] [Info] Number of data points in the train set: 162355, number o
f used features: 21
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.00
0000
[LightGBM] [Info] Start training from score 0.000000
[LightGBM] [Info] Number of positive: 22622, number of negative: 139733
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.012858 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 176
[LightGBM] [Info] Number of data points in the train set: 162355, number o
f used features: 21
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.00
0000
[LightGBM] [Info] Start training from score 0.000000
```

Out[69]:

	Model	CV ROC-AUC mean	std
1	LightGBM	0.830371	0.002340
0	Logistic Regression	0.824694	0.001622
2	XGBoost	0.822745	0.002225
3	Random Forest	0.796803	0.003062
4	Decision Tree	0.592762	0.005850

Interpretation

Cross-validation on the training set shows how stable each model is. Cross-validation shows how stable each model is. LightGBM leads (0.830) by a small but consistent margin over Logistic Regression (0.825) and XGBoost (0.823), about 2 standard deviations (std ≈ 0.002). Logistic Regression and XGBoost are within noise of each other.

All models see the identical, leak-free training matrix. Class weights (or `scale_pos_weight`) make errors on the rare diabetic class more costly, which pushes the models towards higher recall. KNN is left unweighted (it has no class-weight mechanism), and the SVM uses a stratified 30k subsample purely for computational reasons. The Decision Tree is deliberately left unpruned as a baseline, and `probability=True` lets the SVM output probabilities for ROC/PR curves.

Model Evaluation

Because only ~14% of respondents are diabetic, accuracy is not enough. We report **ROC-AUC, PR-AUC, F1, precision and recall for class 1**, the confusion matrices, and the ROC and precision-recall curves on the untouched test set.

In [ ]:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

def evaluate_model(model, X, y, name):
    preds = model.predict(X)
    pred_probs = model.predict_proba(X)[:, 1]
    fpr, tpr, thresholds = roc_curve(y, pred_probs, pos_label=1)
    auc_score = roc_auc_score(y, pred_probs)
    pr_auc = average_precision_score(y, pred_probs)
    precision, recall, pr_thresholds = precision_recall_curve(y, pred_pro
    return {
        'Model': name,
        'Accuracy': accuracy_score(y, preds),
        'Precision': precision_score(y, preds),
        'Recall': recall_score(y, preds),
        'F1-Score': f1_score(y, preds),
        'AUC': auc_score,
        'FPR': fpr,
        'TPR': tpr,
        'PR': pr_auc,
        'Precision Curve': precision,
```

```
        'Recall Curve': recall

    }

results = pd.DataFrame([
    evaluate_model(log_reg, X_test_s, y_test, 'Logistic Regression'),
    evaluate_model(dt, X_test_s, y_test, 'Decision Tree'),
    evaluate_model(rf_classifier, X_test_s, y_test, 'Random Forest'),
    evaluate_model(clf, X_test_s, y_test, 'LightGBM'),
    evaluate_model(xgb, X_test_s, y_test, 'XGBoost'),
    evaluate_model(knn, X_test_s, y_test, 'KNN'),
    evaluate_model(svm_model, X_test_s, y_test, 'SVM')
])

print(results)
```

```
                Model  Accuracy  Precision  Recall  F1-Score    AUC  \
3                 lgb     0.721      0.306   0.793     0.442  0.827
0  Logistic Regression     0.732      0.312   0.763     0.442  0.821
4              XGBoost     0.730      0.310   0.768     0.442  0.821
6                  SVM     0.720      0.302   0.766     0.433  0.804
2        Random Forest     0.857      0.460   0.159     0.236  0.792
5                  KNN     0.834      0.364   0.253     0.299  0.682
1        Decision Tree     0.798      0.296   0.325     0.310  0.600

                                                 FPR  \
3  [0.0, 0.0, 0.0, 2.290058854512561e-05, 2.29005...
0  [0.0, 0.0, 4.580117709025122e-05, 4.5801177090...
4  [0.0, 0.0, 0.0, 2.290058854512561e-05, 2.29005...
6  [0.0, 2.290058854512561e-05, 4.580117709025122...
2  [0.0, 2.290058854512561e-05, 6.870176563537683...
5  [0.0, 0.01330524194471798, 0.01332814253326310...
1  [0.0, 0.11441134037144754, 0.11468614743398906...

                                                 TPR     PR  \
3  [0.0, 0.00014146272457207527, 0.00028292544914...  0.424
0  [0.0, 0.00014146272457207527, 0.00014146272457...  0.404
4  [0.0, 0.00014146272457207527, 0.00084877634743...  0.412
6  [0.0, 0.0, 0.0, 0.00028292544914415053, 0.0002...  0.356
2  [0.0, 0.0, 0.0, 0.00014146272457207527, 0.0001...  0.354
5  [0.0, 0.058424105248267084, 0.0584241052482670...  0.259
1  [0.0, 0.3167350403168765, 0.31715942849059275,...  0.195

                                     Precision Curve  \
3  [0.13932907600126143, 0.1393318222134621, 0.13...
0  [0.13932907600126143, 0.1393318222134621, 0.13...
4  [0.13932907600126143, 0.1393318222134621, 0.13...
6  [0.13932907600126143, 0.1393318222134621, 0.13...
2  [0.13932907600126143, 0.1740175003793435, 0.17...
5  [0.13932907600126143, 0.27192245557350564, 0.2...
1  [0.13932907600126143, 0.29049785190801114, 0.2...

                                        Recall Curve
3  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, ...
0  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, ...
4  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, ...
6  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, ...
2  [1.0, 0.9734050077804498, 0.9734050077804498, ...
5  [1.0, 0.5952751449992927, 0.5952751449992927, ...
1  [1.0, 0.325222803791201, 0.325222803791201, 0....
```

- **LightGBM** has the best ROC-AUC (0.827) and PR-AUC (0.424) and the highest recall (0.793).
- **XGBoost** (F1 0.442, ROC-AUC 0.821, PR-AUC 0.412) and **Logistic Regression** (F1 0.442, ROC-AUC 0.821, PR-AUC 0.404) are essentially tied with LightGBM on F1; the differences are in the third decimal and are not meaningful on a single split.
- **SVM** (precision 0.302, recall 0.766, F1 0.433, ROC-AUC 0.804, PR-AUC 0.356) has a recall similar to Logistic Regression and the boosting models, with slightly lower ranking quality (AUC).
- **Random Forest** (accuracy 0.857, precision 0.460, recall 0.159, F1 0.236) has the highest accuracy, but at the default threshold it misses most diabetic cases. Its accuracy is close to the 0.861 majority-class baseline.
- **KNN** (ROC-AUC 0.682, PR-AUC 0.259) and the unpruned **Decision Tree** (ROC-AUC 0.600, PR-AUC 0.195) have the weakest discrimination.

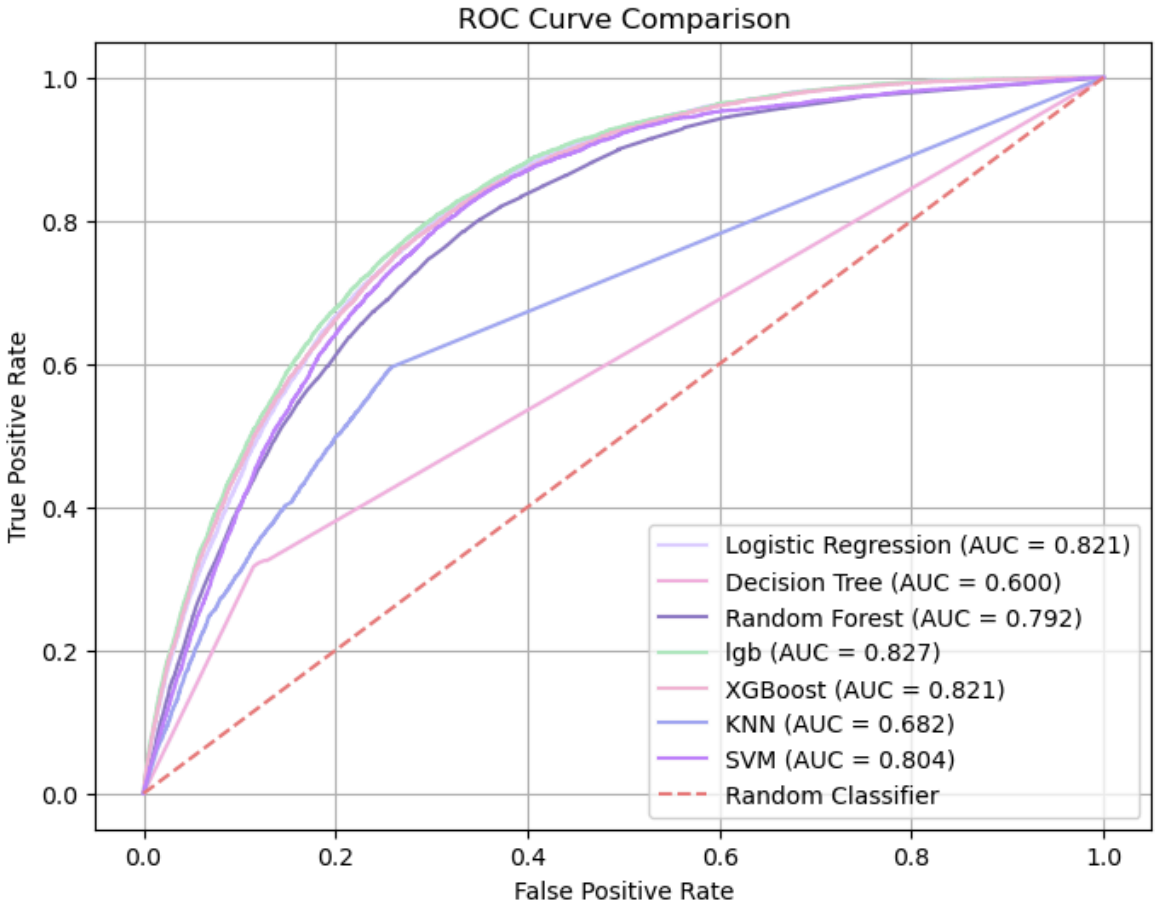
In [30]:
```
from sklearn.metrics import roc_curve

# fpr1, tpr1, thresh1 = roc_curve(y_test, pred_lr[:,1], pos_label=1)
# # fpr2, tpr2, thresh2 = roc_curve(y_test, pred_prob2[:,1], pos_label=1)
# random_probs = [0 for i in range(len(y_test))]
# p_fpr, p_tpr, _ = roc_curve(y_test, random_probs, pos_label=1)
```

In [35]:
```
plt.figure(figsize=(8, 6))
palett =['#DCD0FF', '#F0B2E0', '#8E7CC3', "#AFEABF", '#F4B6D2', "#A1A9F1"
for i, row in results.iterrows():
    plt.plot(
        row['FPR'],
        row['TPR'],
        linestyle='-',
        label=f"{row['Model']} (AUC = {row['AUC']:.3f})", color=palett[i]
    )

plt.plot([0, 1], [0, 1], linestyle='--', color="#E87878", label='Random C
plt.grid()
plt.title('ROC Curve Comparison')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')

plt.legend(loc='best')
plt.show()
```

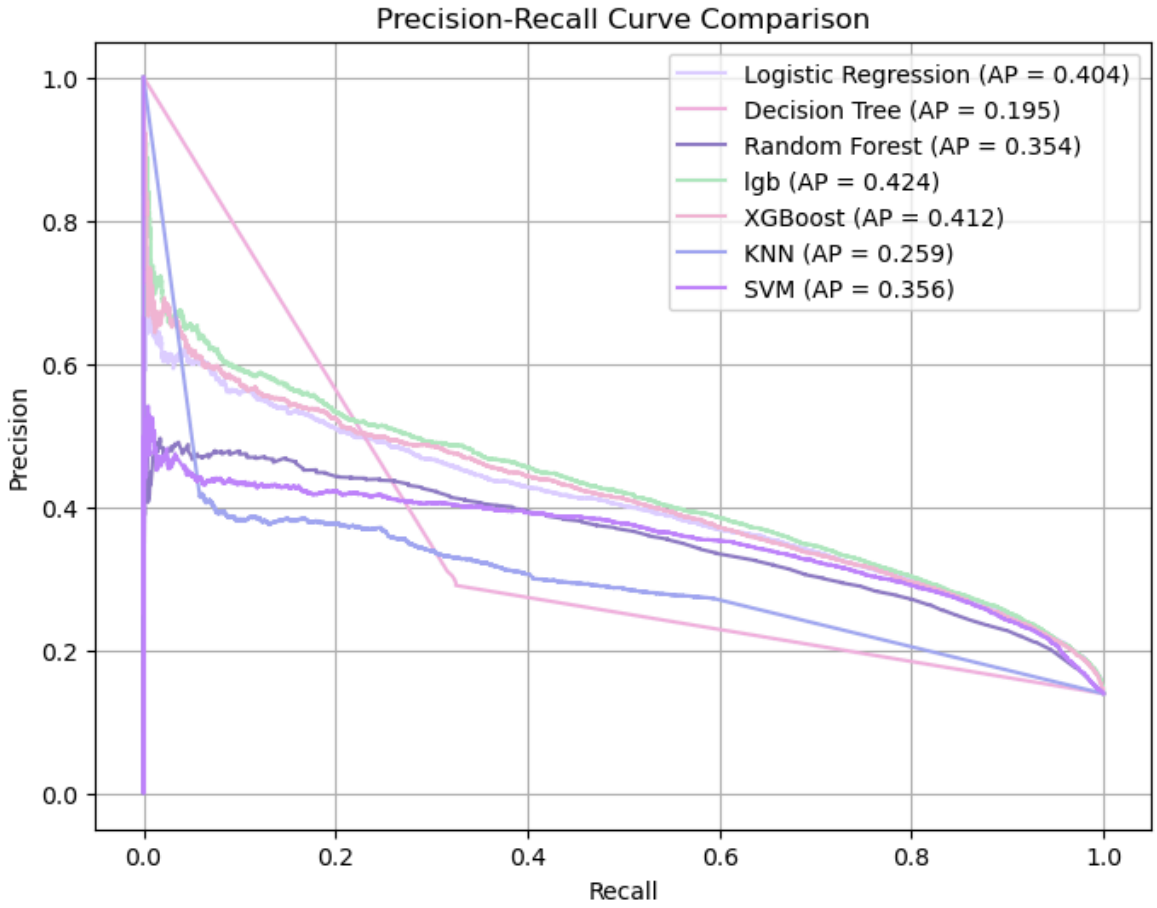


ROC curves

LightGBM (0.827), XGBoost and Logistic Regression (0.822) have almost overlapping curves well above the diagonal, so they rank diabetic and non-diabetic respondents about equally well. SVM (0.804) and Random Forest (0.792) are slightly lower. KNN (0.683) and especially the Decision Tree (0.598) lie close to the diagonal over much of the range, which shows their weak ranking ability.

In [34]:
```
plt.figure(figsize=(8, 6))
palett =['#DCD0FF', '#F0B2E0', '#8E7CC3', "#AFEABF", '#F4B6D2', "#A1A9F1"

for i, row in results.iterrows():
    plt.plot(
        row['Recall Curve'],
        row['Precision Curve'],
        label=f"{row['Model']} (AP = {row['PR']:.3f})", color=palett[i]
    )
plt.grid()
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve Comparison')
plt.legend(loc='best')
plt.show()
```



PR curves

PR curves are more informative than ROC under imbalance. All models beat the 0.139 no-skill baseline, with LightGBM highest (AP 0.424), followed by XGBoost (0.412), Logistic Regression (0.404), SVM (0.356) and Random Forest (0.354).. The PR curve separates the models more than the ROC curve does. Precision falls quickly as recall increases for every model, which means catching more diabetic cases comes at the cost of many false alarms.

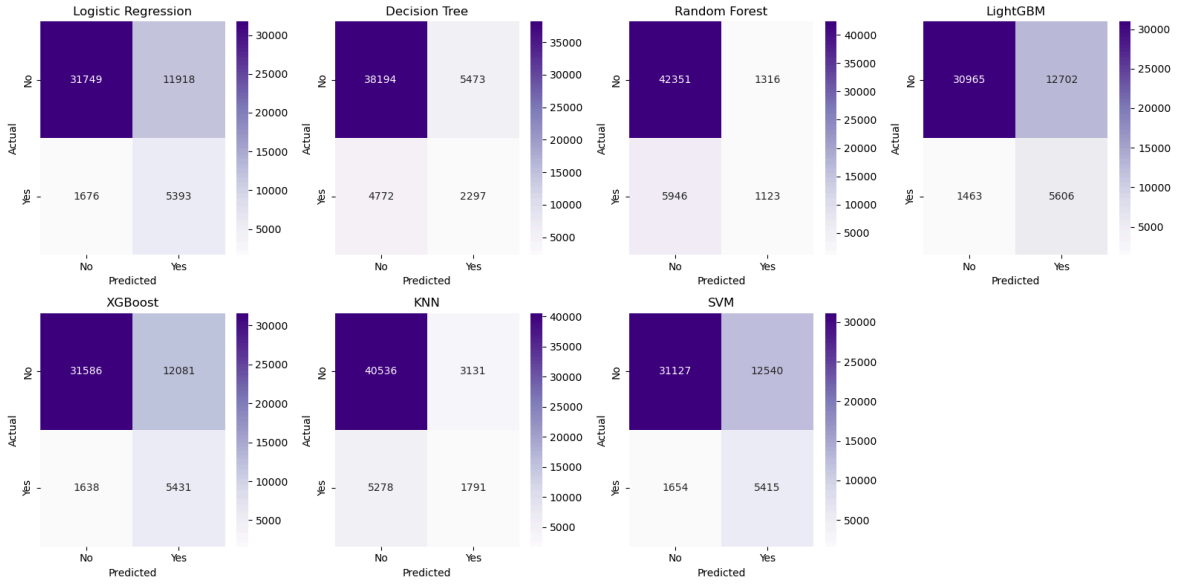
In [73]:
```
cache = {}
def conf_m(model, X, y, name):
    preds =  model.predict(X)
    cm =  confusion_matrix(y, preds, labels=[0, 1])

    sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', xticklabels=['No
    plt.title(name)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')

models = [
    (log_reg, 'Logistic Regression'),
    (dt, 'Decision Tree'),
    (rf_classifier, 'Random Forest'),
    (clf, 'LightGBM'),
    (xgb, 'XGBoost'),
    (knn, 'KNN'),
    (svm_model, 'SVM')
]
plt.figure(figsize=(16, 8))
```

```
for i, (model, name) in enumerate(models, 1):
    plt.subplot(2, 4, i)
    conf_m(model, X_test_s, y_test, name)

plt.tight_layout()
plt.show()
```

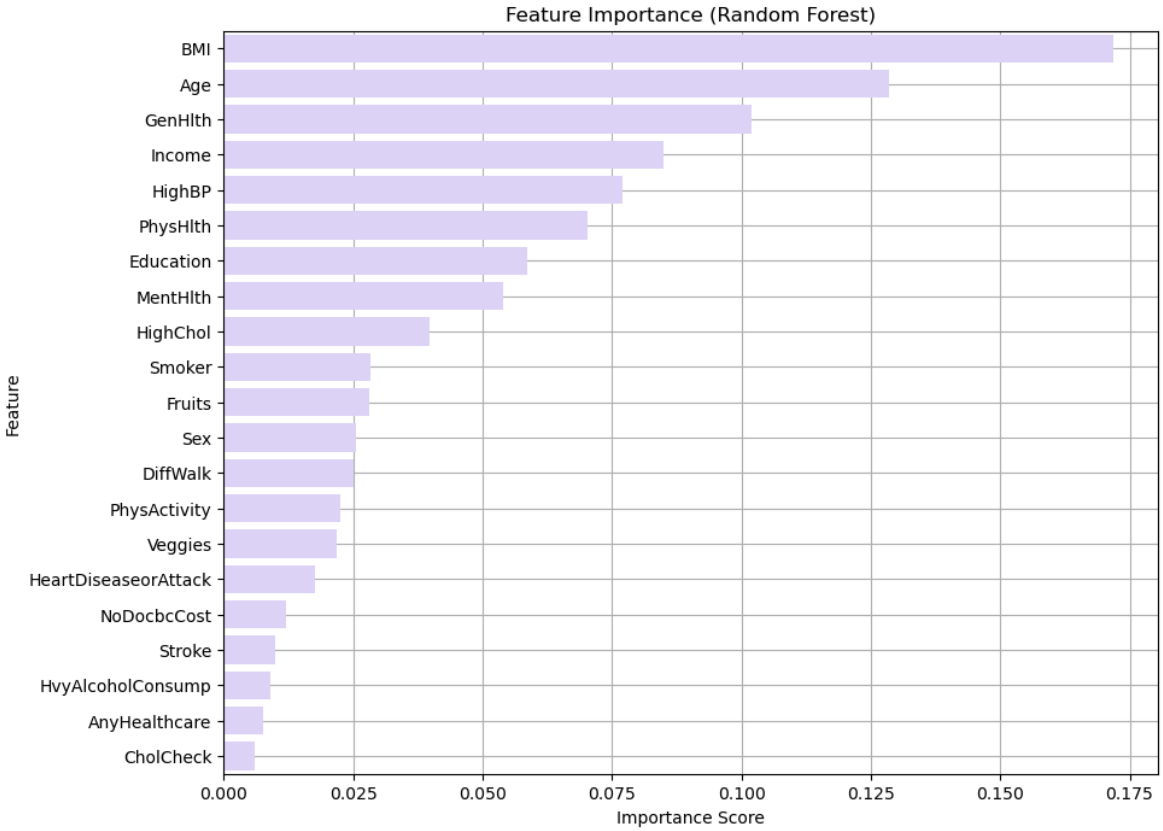


The test set has 7,069 diabetic and 43,667 non-diabetic respondents.

- **LightGBM** catches 5,606 diabetic cases (TP) and misses 1,463 (FN), but flags 12,702 non-diabetic respondents as positive (FP). Precision is 0.31. **XGBoost** catches 5,431 and misses 1,638 (12,081 FP). **Logistic Regression** catches 5,393 and misses 1,676 (11,918 FP).
- **Random Forest** identifies 1,123 diabetic cases, misses 5,946, with 1,316 false positives. Its precision is about 0.45 but its recall is only 0.17.
- **SVM** catches 5,415 diabetic cases and misses 1,654, but flags 12,540 non-diabetic respondents as positive, similar to the other class-weighted models.
- **KNN** catches about 1,791 cases (about 5,278 missed, about 3,131 FP) and the **Decision Tree** catches about 2,297 (4,772 missed, 5,473 FP). Both have much lower recall than LightGBM, XGBoost and Logistic Regression.
- Which model is better depends on the cost of a missed case versus a false alarm. For population screening, LightGBM is preferable because it identifies the most diabetic cases. Where false positives are costly.

In [ ]:
```
importances = pd.Series(rf_classifier.feature_importances_, index=X.colum
importances = importances.sort_values(ascending=False)

plt.figure(figsize=(10,8))
sns.barplot(x=importances.values, y=importances.index, color='#DCD0FF')
plt.grid( zorder=0)
plt.gca().set_axisbelow(True)
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Importance Score")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```



Random Forest ranks `BMI` (0.17), `Age` (0.13), `GenHlth` (0.11), `Income` (0.08) and `HighBP` (0.08) highest, and `CholCheck`, `AnyHealthcare`, `HvyAlcoholConsump` and `Stroke` lowest. Impurity-based importance favours features with many distinct values (BMI, Age, Income), so this is partly a bias of the method.

5. Discussion and Summary of Insights

5.1 Results at a glance

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
LightGBM	0.721	0.306	0.793	0.442	0.827	0.424
XGBoost	0.730	0.310	0.768	0.442	0.821	0.412
Logistic Regression	0.732	0.312	0.763	0.442	0.821	0.404
SVM	0.720	0.302	0.766	0.433	0.804	0.356
Random Forest	0.856	0.460	0.159	0.236	0.792	0.354
KNN	0.833	0.360	0.255	0.298	0.682	0.259
Decision Tree	0.798	0.296	0.325	0.310	0.600	0.195

5.2 Key findings

1. **LightGBM has the best overall ranking performance.** It leads on ROC-AUC (0.827), PR-AUC (0.424) and recall (0.793). XGBoost and Logistic Regression are very close (F1 = 0.442 for all three), so a simple linear model competes with boosting. On cross-validated ROC-AUC the three models are also close, so the recommendation of LightGBM rests on its slightly higher AUC, PR-AUC and recall, not on a large gap.

2. **Accuracy is misleading for imbalanced data.** Random Forest has the highest accuracy (0.856), almost the same as the 0.861 majority-class baseline, yet its recall is only 0.159: at the default 0.5 threshold it predicts "no diabetes" for almost everyone. The class-weighted models (LightGBM, XGBoost, Logistic Regression, SVM) have lower accuracy (0.72–0.73) but find 76–79% of diabetic respondents.

3. **Class weights make the biggest difference to recall.** LightGBM, XGBoost and Logistic Regression use class weights and reach recall of 0.763–0.793. The price is low precision, because the diabetic class is only about 14% of the data.

4. **Precision stays low when recall is high.** LightGBM has precision 0.306 at recall 0.793: about 31% of the respondents it flags are actually diabetic, and the rest are false alarms. The survey contains only self-reported features, with no laboratory values such as blood glucose or HbA1c, which limits how well the classes can be separated.

5. **Decision Tree and KNN discriminate poorly.** The Decision Tree (ROC-AUC 0.600, PR-AUC 0.195) was left unpruned, so it overfits. KNN (ROC-AUC 0.682, PR-AUC 0.259) used k = 3, which is very noisy. Part of their weakness comes from these baseline settings, not only from the algorithms.

6. **Preprocessing choices were checked, not assumed.** Missing values (none), outliers (BMI winsorised at the 1st and 99th percentile of the training set), duplicate rows, contradictory records and variable coding were all examined. Scaling matters for Logistic Regression, KNN and SVM, whereas tree models do not need it.

5.3 Which features matter

Across the EDA, correlation analysis, Random Forest importance, the most relevant variables are general health (`GenHlth`), high blood pressure, BMI, age, high cholesterol, difficulty walking and income.

Diabetes prevalence is associated with poorer self-rated health, hypertension, high cholesterol, higher BMI and older age. Income, education and physical activity also carry useful information, with higher values linked to lower prevalence. These are associations and do not prove causation.

`AnyHealthcare`, `NoDocbcCost` and `Sex` contribute little. The lower diabetes rate among heavy drinkers probably reflects confounding (for example, people diagnosed with diabetes may drink less), not a protective effect. Random Forest importance favours variables with many distinct values (BMI, Age, Income), so its ranking should be read with caution.

5.4 Practical interpretation

The best model depends on the intended use.

- **Population screening:** missing a case is costly and follow-up testing is available, so LightGBM is the best choice because it finds the most diabetic respondents (recall 0.793) and has the best ROC-AUC and PR-AUC. Logistic Regression is a good alternative: its performance is close, and it is easier to interpret.
- **Limited follow-up resources:** a higher decision threshold would reduce false positives and raise precision, at the cost of missing more cases. Random Forest and SVM should not be chosen because of their high accuracy, since at the default threshold they identify very few diabetic respondents.

I recommend LightGBM as the primary model and Logistic Regression when interpretability is a priority.

5.5 Limitations and future work

- The data come from the self-reported BRFSS 2015 survey. They cannot reliably separate type 1 from type 2 diabetes, and they describe associations, not causes.
- Everything was evaluated at the default 0.5 threshold. Choosing the threshold on a validation set (not on the test set) would balance precision and recall better, especially for SVM and Random Forest.
- The SVM was trained on a 30,000-row subsample for computational reasons, so its comparison with the other models is not fully like-for-like.

5.6 Conclusion

A health-indicator survey can separate respondents with different diabetes risk with moderate performance: LightGBM reaches a ROC-AUC of about 0.83. Its precision of 0.306 at a recall of 0.793 means many positive predictions are false alarms, so the model is suitable for flagging people for further testing, not for diagnosis.

The most relevant predictors are general health, blood pressure, BMI, age and other health and socioeconomic indicators. The results also show why accuracy alone is

insufficient for imbalanced data: SVM and Random Forest have high accuracy but miss most diabetic respondents.

Overall, LightGBM gives the strongest performance for identifying potential diabetes cases, and Logistic Regression is a competitive, more interpretable alternative. Both need further validation before any practical screening use.